

A-Evolve: Our Shared Mental Model

Self-Evolving Agents with Frontier Models as Researchers

Hanqing Lu | Internal memo for A-EVO Lab | April 2026

This memo is meant to get us aligned on the frame we’re working inside. A-Evolve treats self-evolution as **AI-as-researcher**: frontier agents and models play the researcher role in the loop that improves agent capability — reading trajectories, proposing modifications, running experiments, and updating both the agent’s *harness* (memory / skill / prompt / workflow) and its *model weights*. Our three algorithms span a 2×2 (harness vs. weights $\times$ single- vs. multi-agent evolver). The evolver is human-designed; once specified, solver evolution is end-to-end automated across every benchmark we run — no per-task prompts, no curated solver data. We’re operating $\sim 10^3 \times$ larger per-experiment than prior self-evolution work; only a handful of labs reach this regime today, and what we’re seeing empirically is qualitatively different from the classical regime. The rest of this doc is the shared model I’d like us to work from.

1. Thesis

The core claim we’re building on — **AI agents and models themselves can play the researcher role in the loop that improves agent capability** — became possible only in the last ~ 12 months, once frontier base models became strong enough to do those researcher tasks end-to-end. The practical consequence for us is that agent self-improvement is no longer bottlenecked by hand-designed training pipelines or curated data, but by **how much researcher capacity we can compose inside the loop**. We’ve scoped this into three algorithms (§2), tested across 8+ agentic and 4+ training benchmarks; the rest of this memo focuses on two emergent findings (§3) and the open questions I think we should push on next (§4).

2. Scope: A 2×2 Design Space

Three algorithms,¹ spanning a two-axis space (*harness vs. model-weight $\times$ single-agent vs. multi-agent evolver*):

	Single-agent evolver	Multi-agent evolver
Improve harness	A-Evolve-Agent-Single	A-Evolve-Agent-Multi
Improve model weights	(intentionally empty — see below)	A-Evolve-Training-Multi

Current leaderboard standings (one evolver across all, solver fully automated, for reference): MCP-Atlas #1 (79.4%) | SkillBench #2 (34.9%) | SWE-bench Verified #5 (76.8%) | Terminal-Bench 2.0 #7 (76.5%) | ARC-AGI +2.2pp.

The A-Evolve framework — one infra, three algorithms. Whether AI-as-researcher generalizes — across harness vs. weights, single- vs. multi-agent evolvers, benchmarks, model families, and compute scales — requires experiment volume no existing framework supports. The framework’s role is **precondition, not convenience**: without it the thesis isn’t testable at scale, only anecdotal. Our repo is open-source at github.com/A-EVO-Lab/a-evolve; the primitives — evolver state, artifact representation, modification operators, trajectory evaluation — are factored so that new evolver variants can be prototyped in days and different evolver designs can be compared apples-to-apples on shared scaffolding. Internally, a single abstraction already covers harness-single, harness-multi, and training-multi; the public repo currently hosts predecessor evolvers and external contributions (including a third-party port of GEPA), with A-Evolve-Agent-Single releasing imminently. The fact that our primitives absorbed an externally-designed algorithm (GEPA) without modification is the first empirical sign they’re at the right level of granularity — the broader ambition, to be explicit about it, is that they do for evolver research what PyTorch’s abstractions did for NN research.

A-Evolve-Agent-Single. Single-agent evolver that iteratively modifies an agent’s harness (memory / skill / prompt / workflow) for moderate-scale agentic tasks. Evaluated on 8 benchmarks — ARC-AGI, CL-bench, OSWorld, WebArena, SkillBench, MCP-Atlas, SWE-bench Verified, Terminal-Bench 2.0 — with measurable gains on all (top-line results in the callout above).

A-Evolve-Agent-Multi. Multi-agent evolver for tasks whose *optimal solver must itself be a complex multi-agent system* — e.g., prediction-market tasks like FutureX, where solving well requires a solver that researches, forecasts, debates, and aggregates under uncertainty. Composing such a solver from a single-agent baseline requires harness modifications at a scale a single-agent evolver cannot propose (promoting the solver itself into a multi-agent orchestration, coordinating cross-agent state); scoped human-in-the-loop verification is introduced at steps the evolver flags as high-uncertainty.

¹Technical papers are in preparation for each algorithm, plus a dedicated writeup of the role-conditioned capability finding (§3).

A-Evolve-Training-Multi. Multi-agent evolver at the model-weight layer. The claim behind the “multi” here is about research, not convenience: effective model-weight evolution decomposes into distinct research roles (hypothesis generation, ablation design, trajectory filtering, result critique, artifact integration), and a single-agent evolver can’t cover this decomposition at quality. Target benchmarks: MLE-bench, Nemo-30B reasoning challenge, Parameter Golf, and large-scale Nova downstream fine-tuning (internal).

On the empty cell (harness $\times$ training with a single-agent evolver). This is deliberate and itself a research claim: harness-level modification fits a single-agent evolver because the search space is structured by familiar code/data abstractions, while model-weight evolution requires the multi-agent decomposition above. I don’t claim this is permanent — whether training-single is reachable with the right scaffolding is one of the questions I think we should push on next (§4).

3. Two Emergent Findings

Finding 1 — Harness-level verification is where A-Evolve-Agent-Single reliably converges. Across SWE-bench, Terminal-Bench 2.0, ARC-AGI, and MCP-Atlas, A-Evolve-Agent-Single consistently evolves harnesses that install verification primitives at the harness layer. The underlying model already has verification capability; the non-trivial claim — and the one I want us aligned on — is that **harness-layer verification is what makes the final agent stable**, not model-level per-call verification alone:

- verification frequency in the evolved harness rises 24% $\rightarrow$ 78% across evolution rounds;
- Terminal-Bench 2.0 unstable-task pass rate rises 33% $\rightarrow$ 67%;
- 3 of 5 skills A-Evolve-Agent-Single converges on for MCP-Atlas are verification primitives.

Structurally this parallels where o1-era models install verification (inside a single problem’s reasoning trace), with the difference that harness-layer verification *persists across tasks* and doesn’t need to be re-derived per problem. What A-Evolve-Agent-Single is reliably good at is *building stable harnesses*, and harness-layer verification is a recurring attractor of that stability. This is the mental frame I’d like us to use when reading evolution traces.

Finding 2 — Role-conditioned capability asymmetry. Varying evolver and solver independently across three frontier-class families (Claude, Qwen, Kimi — the set available via AWS Bedrock at the per-experiment compute we need; GPT and Gemini weren’t accessible at this scale), we hit a counter-intuitive result: **a strictly stronger evolver does not lift a weak solver**. Evolved harnesses escalate long-context and tool-use fidelity demands past what a weak solver can execute — the harness becomes too strong for the solver. This implies evolver and solver are **distinct post-training targets**, not the same capability viewed through different prompts. For RFT-style post-training the concrete implication is that the data distribution that improves a model-as-solver is not the one that improves a model-as-evolver, and the right setup is likely role-conditioned co-training rather than a single general objective. Worth internalizing when we’re designing future training runs.

Position paper for shared context: *Agentic Evolution is the Path to Evolving LLMs* ([arXiv:2602.00359](https://arxiv.org/abs/2602.00359)).

4. Open Questions I Think We Should Push On Next

1. **The harness-to-model bridge as a first-class operator inside the loop.** A-Evolve-Training-Multi currently treats post-training as one pluggable operator. The sharper question: can the evolver-as-researcher frame turn post-training itself into a *search problem* — evolver proposes which evolved trajectories to SFT on, what reward signals to attach, which parameter subsets to touch, and the loop evaluates post-trained variants as just another modification? Early signal is positive for narrow cases; the general version is a serious research program and I’d like us to scope it explicitly.
2. **Where the frontier-evolver loop saturates.** Classical self-evolution saturates because evolver and solver share a learner’s capacity. In our setup the evolver is frontier-class, so the ceiling moves up — but to what? My working hypothesis is that the loop saturates when the evolver’s proposals require research judgment the frontier model cannot yet produce (novel benchmark design, load-bearing architectural bets, calibrated risk-taking on ablation cost). Mapping this ceiling empirically is directly tied to what the next frontier model needs to be able to do — and therefore worth being deliberate about.
3. **Multi-agent evolvers as a test of the multi-agent-reasoning thesis.** A-Evolve-Agent-Multi and A-Evolve-Training-Multi are both “a multi-agent system doing research on another agent.” This is a clean sub-case of the broader multi-agent-reasoning question: what does a small society of frontier researchers produce that a single one does not? Early evidence from our runs suggests multi-agent evolvers produce more *structural* modifications (willing to redesign solver topology, willing to abandon dominant baselines), but we don’t yet have a principled

characterization of when multi-agent strictly dominates single. Good candidate for a focused study.

4. **Scoped human-in-the-loop as a principled element, not a fallback.** Both “multi” variants include scoped human verification. Open question: can we characterize the *minimal* human input that shifts the loop from saturating to non-saturating? This is a clean way to probe what human judgment supplies that frontier models cannot yet — without having to commit to an answer in advance.